

QUERY OPTIMIZATION

Range Partitioning

Sample customers Table with Age-Based RANGE Partitioning

```
-- Create the database
CREATE DATABASE customer_db;

-- Select the database
USE customer_db;

-- Create the partitioned customers table
CREATE TABLE customers (
  customer_id INT NOT NULL,
  customer_name VARCHAR(100) NOT NULL,
  age INT NOT NULL,
  email VARCHAR(100),
  PRIMARY KEY (customer_id, age)
)
PARTITION BY RANGE (age) (
  PARTITION p_young VALUES LESS THAN (18),
  PARTITION p_adult VALUES LESS THAN (30),
  PARTITION p_middle_age VALUES LESS THAN (50),
  PARTITION p_senior VALUES LESS THAN (65),
  PARTITION p_elderly VALUES LESS THAN MAXVALUE
);
```

Important MySQL detail: when using RANGE partitioning, the partitioning column generally needs to be included in every unique key, including the primary key. That's why the example uses:

```
PRIMARY KEY (customer_id, age)
```

The primary key normally identifies a record uniquely. In MySQL partitioned tables, the partitioning column must be included in every unique key. Therefore, when *age* is used for RANGE partitioning, *age* must also be included in the primary key.

Partition	Age Range	Description
p_young	< 18	Customers below 18
p_adult	18–29	Young adults
p_middle_age	30–49	Middle-aged customers
p_senior	50–64	Senior customers
p_elderly	65+	Elderly customers

MAXVALUE in MySQL RANGE partitioning means "any value greater than or equal to the previous partition's boundary." It acts as the upper limit for all remaining values.

For example, if you insert: INSERT INTO customers VALUES

```
INSERT INTO customers (customer_id, customer_name, age, email)
VALUES
(1, 'Alice', 15, 'alice@email.com'),
(2, 'Bob', 25, 'bob@email.com'),
(3, 'Charlie', 35, 'charlie@email.com'),
(4, 'David', 55, 'david@email.com'),
(5, 'Emma', 70, 'emma@email.com'),
(6, 'Loree', 17, 'loree@email.com'),
(7, 'Ethan', 28, 'ethan@email.com'),
(8, 'Elysse', 45, 'elysse@email.com'),
(9, 'Michael', 62, 'michael@email.com'),
(10, 'Euri', 80, 'euri@email.com');
```

MySQL automatically places the records into the appropriate partitions:

```
p_young    → Alice (15)
p_adult    → Bob (25)
p_middle_age → Charlie (35)
p_senior   → David (55)
p_elderly  → Emma (70)
```

Viewing the Partitions

You can check the partitions using:

```
SELECT
    TABLE_NAME,
    PARTITION_NAME,
    TABLE_ROWS
FROM INFORMATION_SCHEMA.PARTITIONS
WHERE TABLE_NAME = 'customers';
```

You can also retrieve data from a specific partition:

```
SELECT * FROM customers PARTITION (p_adult);
```